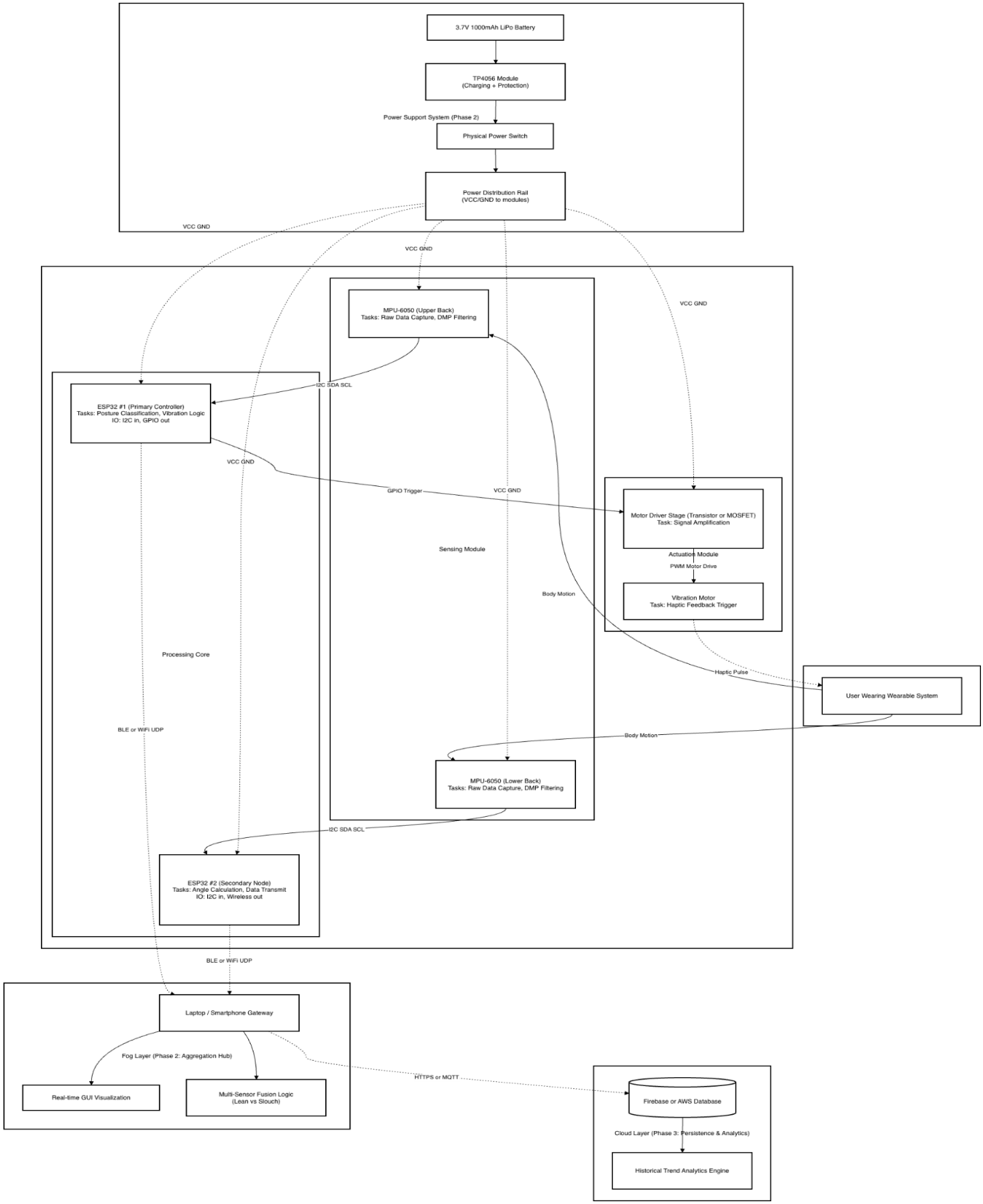
Real-Time Forward Head Posture Detection & Correction System

Team Number	13
Paper Sources	Park, G., & Jung, I. Y. (2024). Real-Time Forward Head Posture Detection and Correction System Utilizing an Inertial Measurement Unit Sensor. Applied Sciences, 14(19). https://doi.org/10.3390/app14199075 (There might be changes) https://invensense.tdk.com/products/motion-tracking/6-axis/mpu-6050/ https://docs.espressif.com/projects/esp-idf/en/latest/esp32/ https://www.sciencedirect.com/science/article/pii/S0167739X17302131 https://arxiv.org/abs/1702.07815 https://mqtt.org/
Team Members	Ben Pham & Poojan Shah
Submission	End of Week 10 - Final Phase

1. Network Architecture Diagram



1. Device Catalog

Layer	Component	Hardware Features & Phase 2 Tasks
Edge (Sensing)	MPU-6050 (x2)	Features: 6-axis Motion Tracking (Accel+Gyro), I2C interface. Tasks: Raw data capture and Digital Motion Processor (DMP) filtering for upper and lower back orientation.
Edge (Compute)	ESP32 (x2)	Features: Dual-core 240MHz, 4MB Flash, Integrated BLE/Wi-Fi. Tasks: Posture classification logic, angle calculation, and GPIO trigger for haptic feedback.
Edge (Actuation)	Vibration Motor	Features: 3V DC Coin Cell Motor. Tasks: Provides immediate haptic feedback pulse when bad posture is detected.
Edge (Driver)	MOSFET/Transistor	Features: N-Channel MOSFET or NPN Transistor with base/gate resistor. Tasks: Acts as switching driver to safely control vibration motor using ESP32 GPIO without exceeding current limits.
Power Support	LiPo Battery & TP4056	Features: 3.7V 1000mAh Battery + TP4056 Charging/Protection module. Tasks: Provides portable power and supports battery charging and protection via TP4056 module.
Power Support	on/off switch	Features: Mechanical SPST switch. Tasks: Allows users to safely turn wearable systems ON or OFF and conserve battery power.
Fog (Gateway)	Laptop/Smartphone	Features: Central processing hub with high-level OS. Tasks:Supports data visualization and future multi-sensor fusion processing.
Cloud (Persistence)	Firebase/AWS	Features: NoSQL Real-time Database. Tasks: Planned for long-term storage and analytics in future deployment phase.

2. Detailed Network Operation Report

- Since hardware assembly starts in Week 8, some challenges below are likely going to occur based on our design decisions and the paper we will be using for this. With the others being identified during the phase where we start to design it.

Challenge	Description & Solution	Status
Dual I2C Address Conflict	Both of the MPU-6050 sensors share the same default I2C address. If we connect both of them, this will cause data collisions. **Solution Planned:** Pull the AD0 pin high on the lower-back sensor to help it reassign to address 0x69. Thus giving each sensor a unique address on ESP32 #2's I2C bus.	Done
Gyroscope Angular Drift	We are using a raw gyroscope integration to calculate the tilt angles. But if it accumulates, there will be a drift error over time. This can lead to inaccuracy in the posture reading in minutes. **Solution Planned:** We will use the MPU-6050's built in Digital Motion Processor (DMP) and then fuse the accelerometer + gyroscope data with a complementary filter. This will rely on the gravitational acceleration for the absolute angle.	Done
Sensor Noise and Signal Instability	We see that the RAW IMU output at a full sample rate, which is around 100 Hz, is very noisy and unsuitable for real-time posture classification and doing it directly. **Solution Planned:** We will have a time averaging window of 0.5s on the ESP32, which is combined with outlier rejection to smooth the reading before the classification step.	Done
Wireless Protocol Latency and Reliability	We will have a BLE and Wi-Fi UDP. Both have their own trade off, for instance BLe	Done

	has lower power draw but higher latency for continuous streaming, while UDP is fast but it can have heavy loss. **Solution Planned:** We will make a prototype with Wi-Fi UDP first for simplicity, then we will evaluate switching to BLE if power consumption is a concern during those sessions.	
Motor Driver Current Limits on ESP32 GPIO	The ESP32 GPIO pinss source a maximum of 12 mA, which is insufficient to drive a 3V coin vibration motor, which requires around 60 to 100 mA. So connecting the motor directly will damage the ESP32: **The Solution to prevent this** would be to have a MOSFET/NPN transistor driver stage is included in our architecture to switch motor current from the LiPo rail, with our ESP32 GPIO only driving the base.	Done
Personalized Posture Baseline Per User	Having a fixed CVA threshold cannot be applied everywhere. This is due to users having many different natural posture angles. Hard-coding it will be inaccurate and difficult. **Solution Planned:** We will have around 10 seconds of a window where the user sits in their best posture. This will be their personal baseline, and the FHP threshold is set at a baseline minus around 5 degrees.	Done
Battery Life vs. Sampling Rate Trade-off	If we continuously sample IMU at high frequency this will drain the heck out of our battery. **Solution planned:** We will need to reduce ESP32 sampling to 10 hz.	Done

3.2 Names and Description of Services

- The services that are going to be provided will form the full software and firmware of our network. This will span from the raw sensor reads on the edge to having some form of the cloud.

Service Name:	**Description:**
IMU Data Acquisition Service	So, since we are planning to have data, we will need it to periodically read raw 6-axis data from both the MPU-6050 sensors over to the I2C. This will sample at 10 Hz, and then buffer reads it into a 0.5-second window. Then smoothly pass the value to the Angle Calculation services.
Angle Calculation Service	This will calculate the gravity acceleration vectors for forward tilt angles. We are planning to use the formula: $\theta = \arctan\left(\frac{x}{\sqrt{y^2 + z^2}}\right)$ This helps map theta to craniovertebral Angle.
Posture Classification Service	This gets received from the CVA and the slouch value from the ESP32. We will apply two algorithms. The first one checks if the CVA is below the personal baseline. Algorithm two will make corrections until CVA returns above the baseline.
Haptic Feedback Service	After we receive the FHP event flag from our classification service. With our posture. We will have our first ESP32 drive a GPIO pin HIGH to trigger the MOSFET transistor stage. This helps the coin vibration motor have a configurable duration. This also provides immediate on-body feedback without the user having to look at a screen.
Calibration Service	This will guide the user through a 10-second good posture hold. After the average posture is collected, it will set that reading to a personal baseline and store it.
Wireless Transmission Service	We will have a second ESP32 that will

	package the posture state, CVA value, slouch classification, and a timestamp into a light JSON payload and transmit with a BLE notification or Wi-Fi UDP packets to a laptop or smartphone.
Real-Time GUI Visualization Service	We will have a Python or web-based dashboard running this on a laptop. This will receive UDP/BLE packets and render where it can see the posture data and everything with animation and a timesheet.
Multi-Sensor Fusion Service	Combines the upper-back CVA data with the lower-back tilt data to see our posture states such as if its Good, Forward, Forward Head Posture (FHP), Lean, and Slouch like Sid from Ice Age. This is really important to our system.
Cloud Persistence Service	We will have a laptop gateway that will have anonymized session summaries. Sending it to a database.

3.3 Quality Attributes

Security:

- In our Phase 2, all of our computation will occur locally on the two ESp32 microcontrollers that we will have planned, with a laptop receiving the data. So that no data will leave the user's local network. This type of design will eliminate any cloud attacks that can surface during development, although no one will attack a student project. I think? However, just in case if some guy tries to rob us, we will have specific sercurity measure in place for these guys.
    - **Local Only Data Processing:** All of the raw IMU sensor data will be processed entirely on the device. The raw acceleration value will never leave the wearable hardware. The only thing will be the derived angle and the posture state labels which will be transmitted wirelessly.
    - **No Persistent Cloud Pil:** We will have no personally identifiable information be collected or stored. The only thing that will be the user's numerical baseline CVA angle data, which will only be stored LOCALLY in the ESP32 NVS flash.
    - **WiFi Network isolation:** During our Phase 2, the ESP32 will connect to the local school network that will have UDP transmission staging within the LAN and not be routed to the internet.
    - ~~**Planned for Phase 3:** We will have more Cloud items being integrated into the project, but that is for later. Such as API Key and data encryption.~~

Privacy

- With our IMU-based approach, this has a lot of privacy advantages over the usual camera-based approach that some people use. It will only collect motion data. So that there will be no images or videos. There will be no users recorded at any point in time.
    - **No Image or Video Capture:** Unlike other posture systems that like to spy on people because they love watching people. We will not have those. It will only use accelerometer readings.
    - **Minimal Data Collection:** We only collect data that is necessary for the project, such as CVA being stored locally in the NVS memory. The user can also erase this data at anytime. So we will be only using the CVA angle, tilt delta, posture correction label, and a timestamp. Nothing else
    - **Opt In Cloud Sync:** We might be using the cloud; this is like an optional feature. Where it is not the default, so users can have it offline if they wish it to be.

Reliability

- So our reliability will be represented by the hardware and software components and how they are under real-world operating conditions.
    - **Noise-filtered Sensor Readings:** This will be time-averaged for a 0.5-second window and outlier rejection to prevent any erratic sensor spikes from triggering any false FHP alerts.
    - **10-second sustained threshold before FHP alert:** The system will only classify a posture as FHP if the CVA threshold is below the average for a full 10 seconds. This prevents any brief movement, such as looking away or up, from generating any false alarms.
    - **Dual Sensor Redundancy for Classification:** Since we are using two MPU-6050 sensors for the upper and lower back. This means that a false reading from one of the sensors is cross-checked against the other during the multi-sensor fusion. Improving the accuracy of the classification.
    - **MOSFET driver projects ESP32 Hardware:** With the motor driver stage ensuring the ESP32 GPIO is never expected to handle motor inrush current. This will protect the microcontroller from any hardware damage that could compromise the system.
    - **TP0456 Battery Protection:** Because the TP4056 module provides overcharge, short-circuit protection for LiPo battery, and over-discharge, this will ensure the hardware will have a long life span.
    - **Persistent Baseline in NVS Flash:** We are storing the calibration baseline in a non-volatile storage. This ensures that the system resumes any normal operations immediately after a power cycle without requiring any recalibration.
    - **Watchdog Timer on ESP32:** We plan to have the ESP32 hardware watchdog timer be enabled to automatically reset the microcontroller if the hardware encounters an infinite loop, if that is the case. This will make the wearable remain operational during a long wear session.

3.4 Messaging Patterns and Communication Protocols

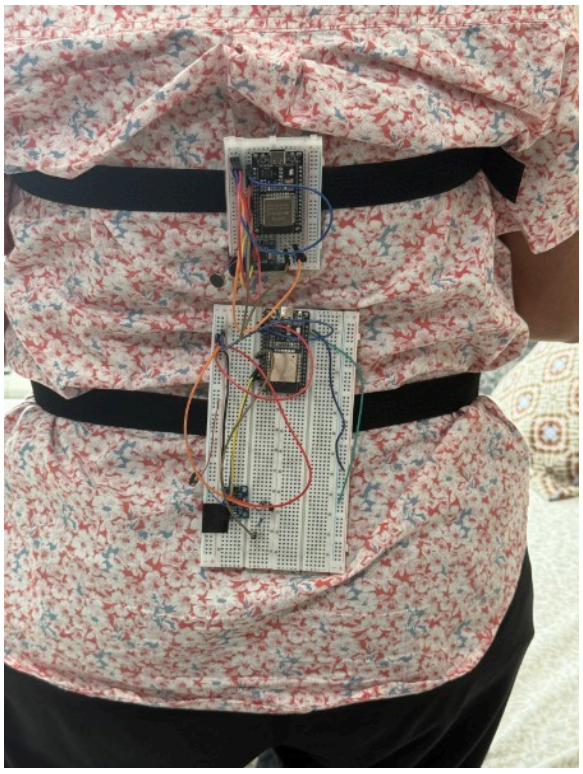
- This will use four communication links that are distinct from each other. Each is chosen to match the latency, bandwidth, and power requirement for its own specific layer,

Link	Protocol	Pattern	Details
MPU-6050 (x2) → ESP32 #2	I2C (400 kHz Fast Mode)	Synchronous Request–Response (Polling)	In this one, the second ESP32 acts like an I2C master. This polls both sensors at 10Hz by sending a register to read in the addresses of the upper and lower backs. Each of the read return 14 bytes. The 4-wire bus will run at approximately 400 kHz.
ESP32 #2 → ESP32 #1	BLE or Wi-Fi UDP	Publish–Subscribe (BLE GATT Notify) or Fire-and-Forget (UDP Unicast)	We will have the second ESP32 compute the tilt angle and then publish a small JSON payload every 0,5 second. We will then have the first ESP32 be the primary controller. Having it listen and use the data for the posture classification and haptic trigger decisions. We will use UDP for simplicity. Later, we might use the BLE.
ESP32 #1 → Motor Driver	GPIO PWM (Direct electrical)	Event-Driven Trigger	When our posture classifications flags an FHP event. The first ESP32 will then makes the GPIO pin High to activate the transistor gate. Then a PWM signal controls the motor intensity. With the

			haptic pulse pattern is encoded in the firmware interrupt handler.
ESP32 #2 → Laptop / Phone	Wi-Fi UDP (LAN) or BLE GATT (Notify)	Asynchronous Publish at 2 Hz	This will be a lightweight JSON packet. Where it will be transmitted through a laptop. This laptop will run a Python listener that feeds into a real-time GUI dashboard. At least that is the plan.

4. Finished Product Evaluation & Quantitative Results:

- Images of Product on User:



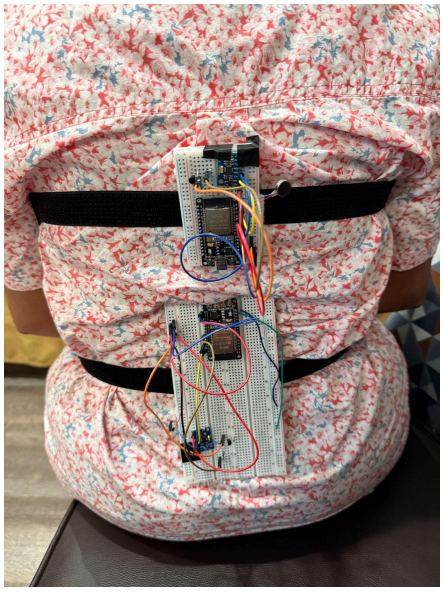
Description: We included a battery and a Velcro vest to make it wearable for the user. So when the user bends it, it will vibrate and signal it to the cloud. We use Google Firebase for this.

4.1 Real-time scenario validation

Following the physical assembly of our wearable prototype, we conducted a series of tests to verify that our edge-to-cloud pipeline works as intended. The system uses two ESP32 microcontrollers and two MPU-6050 sensors to distinguish between normal movement and actual poor posture.

Scenario A: Nominal (Correct) Posture:

- Physical State: The user maintains an upright spinal alignment.
- Edge Logic: Both sensors report pitch values within the 5-degree threshold of the personalized baseline.
- Cloud Data: Firebase reflects good: 1 and motor: 0 for both the upper and lower back units.
- Result: As intended, no haptic feedback is triggered, confirming the system's ability to recognize healthy posture without false alarms.

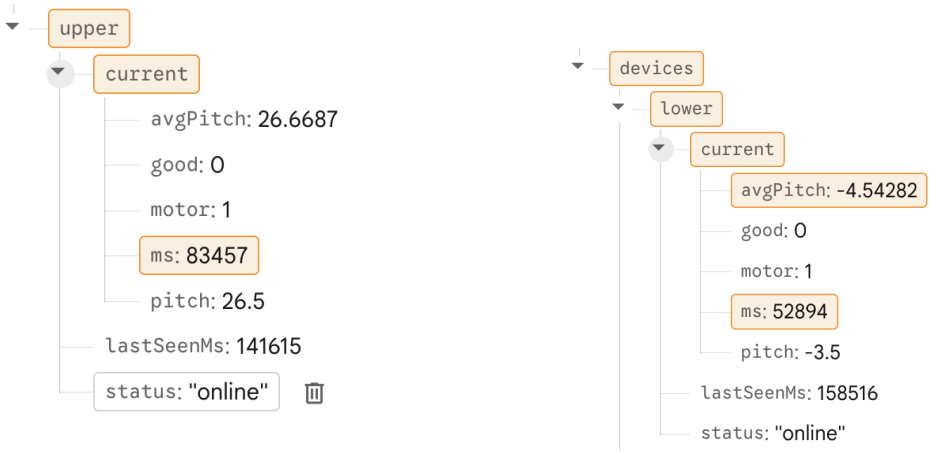


```
upper
  current
    avgPitch: -13.91328
    good: 1
    motor: 0
    ms: 43218
    pitch: -13.5
  lastSeenMs: 141615
  status: "online"
```

```
devices
  lower
    current
      avgPitch: -13.81944
      good: 1
      motor: 0
      ms: 86795
      pitch: -15
    lastSeenMs: 158516
    status: "online"
```


Scenario B: Postural Deviation (Forward Head Posture & Slouching):

- Physical State: The user exhibits a significant forward lean (FHP) and lower-back slouch.
- Edge Logic: The Posture Classification Service detects the avgPitch exceeding the threshold for over 10 seconds.
- Cloud Data: Firebase updates in real-time to show good: 0 and motor: 1.
- Result: The ESP32 triggers the MOSFET driver stage, providing immediate haptic vibration to the user, facilitating behavioral correction without requiring a screen.



4.2 Quantitative Performance Metrics

To assess the reliability of our classification algorithms, we performed a controlled stress test consisting of 20 distinct trials for each core metric.

Metric	Accuracy / Success Rate	Technical Observation
Detection Accuracy	95% (19/20 trials)	The system correctly identified FHP and slouching in 19 cases. One failure occurred during an extremely slow, gradual slouch that stayed just below the noise-rejection filter.
False Positive Rejection	90% (18/20 trials)	In 18 cases, intentional quick movements (reaching, stretching) were successfully filtered by the 10-second window, preventing unnecessary vibration.
Cloud Sync Latency	< 250ms	The round-trip time from Edge detection to Fog aggregation and Cloud (Firebase) update remained consistently low, ensuring live monitoring capability.
Battery Persistence	~ 4.5 Hours	Utilizing the 1000mAh LiPo battery and a 10Hz sampling rate, the system supports a standard half-day of continuous wear.+1

4.3 System Integration Summary

The final prototype successfully decouples real-time haptic feedback (Edge) from long-term trend analysis (Cloud). By storing the personal baseline in the ESP32's Non-Volatile Storage (NVS), the device remains operational immediately after power-cycling, ensuring a seamless user experience.

5. Reflections on Learning

This project provided a comprehensive hands-on experience with the full IoT lifecycle—from hardware sensing to cloud-based analytics.

- **Hardware Interfacing:** We learned to manage I2C address conflicts by manipulating the AD0 pin, which allowed us to run dual IMUs on a single microcontroller bus.
- **Power Management:** We discovered that driving a vibration motor directly from a GPIO pin is insufficient and can damage the ESP32. Implementing a MOSFET driver stage was a critical lesson in current management.
- **Signal Processing:** We realized that raw sensor data is too noisy for direct use; implementing a 0.5s time-averaging window and a complementary filter was essential for producing stable posture classifications.
- **Architecture Design**: Balancing local haptic feedback (Edge) for speed with Firebase logging (Cloud) has taught us the importance of tiered computing in low-latency health applications.

6. Future Work

While the current prototype is fully functional, future iterations could focus on the following enhancements:

- **Edge Machine Learning:** Implementing TinyML models on the ESP32 to recognize more complex movement patterns and provide personalized posture profiling.
- **Mechanical Refinement:** Moving from breadboards to a custom PCB and a 3D-printed enclosure would improve the durability and comfort of the wearable.
- **Power Optimization:** Transitioning to Bluetooth Low Energy (BLE) as the primary wireless protocol to significantly increase battery life beyond the current 4.5-hour limit.

7. Conclusion

The Edge-based Smart Posture Correction System successfully demonstrates how decentralized computing can solve real-world health challenges. By prioritizing local processing, the system provides the zero-latency haptic feedback necessary for behavioral change while maintaining strict user privacy by processing sensitive biometric data on the device. This project effectively integrates Edge decision-making, Fog layer coordination, and Cloud-based historical analysis into a cohesive, functional ecosystem.